

WEEK-3

8-PUZZLE-A*

```
import heapq
from termcolor import colored

class PuzzleState:
    def __init__(self, board, parent, move, depth, cost):
        self.board = board
        self.parent = parent
        self.move = move
        self.depth = depth
        self.cost = cost

    def __lt__(self, other):
        return self.cost < other.cost

def print_board(board):
    print("+---+---+---+")
    for row in range(0, 9, 3):
        row_visual = "|"
        for tile in board[row:row + 3]:
            if tile == 0:
                row_visual += f" {colored(' ', 'cyan')}} |"
            else:
                row_visual += f" {colored(str(tile), 'yellow')}} |"
        print(row_visual)
    print("+---+---+---+")

goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]

moves = {
    'U': -3,
    'D': 3,
    'L': -1,
    'R': 1
}

def heuristic(board):
    distance = 0
    for i in range(9):
        if board[i] != 0:
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(board[i] - 1, 3)
            distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def move_tile(board, move, blank_pos):
    new_board = board[:]
    new_blank_pos = blank_pos + moves[move]
    new_board[blank_pos], new_board[new_blank_pos] = new_board[new_blank_pos],
    new_board[blank_pos]
```

```

    return new_board
def a_star(start_state):
    open_list = []
    closed_list = set()
    heapq.heappush(open_list, PuzzleState(start_state, None, None, 0,
    heuristic(start_state)))

    while open_list:
        current_state = heapq.heappop(open_list)

        if current_state.board == goal_state:
            return current_state

        closed_list.add(tuple(current_state.board))

        blank_pos = current_state.board.index(0)

        for move in moves:
            if move == 'U' and blank_pos < 3:
                continue
            if move == 'D' and blank_pos > 5:
                continue
            if move == 'L' and blank_pos % 3 == 0:
                continue
            if move == 'R' and blank_pos % 3 == 2:
                continue

            new_board = move_tile(current_state.board, move, blank_pos)

            if tuple(new_board) in closed_list:
                continue

            new_state = PuzzleState(new_board, current_state, move,
current_state.depth + 1, current_state.depth + 1 + heuristic(new_board))
            heapq.heappush(open_list, new_state)

        return None

def print_solution(solution):
    path = []
    current = solution
    while current:
        path.append(current)
        current = current.parent
    path.reverse()

    for step in path:
        print(f"Move: {step.move}")
        print_board(step.board)

```

```

initial_state = [1, 2, 3, 4, 0, 5, 6, 7, 8]
solution = a_star(initial_state)

if solution:
    print(colored("Solution found:", "green"))
    print_solution(solution)
else:
    print(colored("No solution exists.", "red"))

```

Solution found:

Move: None

```

+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 |   | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+

```

Move: R

```

+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 |   |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+

```

Move: D

```

+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
| 6 | 7 |   |
+---+---+---+

```

Move: L

```

+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
| 6 |   | 7 |
+---+---+---+

```

Move: L

```

+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
|   | 6 | 7 |

```

+---+---+---+

Move: U

+---+---+---+

| 1 | 2 | 3 |

+---+---+---+

| | 5 | 8 |

+---+---+---+

| 4 | 6 | 7 |

+---+---+---+

Move: R

+---+---+---+

| 1 | 2 | 3 |

+---+---+---+

| 5 | | 8 |

+---+---+---+

| 4 | 6 | 7 |

+---+---+---+

Move: D

+---+---+---+

| 1 | 2 | 3 |

+---+---+---+

| 5 | 6 | 8 |

+---+---+---+

| 4 | | 7 |

+---+---+---+

Move: R

+---+---+---+

| 1 | 2 | 3 |

+---+---+---+

| 5 | 6 | 8 |

+---+---+---+

| 4 | 7 | |

+---+---+---+

Move: U

+---+---+---+

| 1 | 2 | 3 |

+---+---+---+

| 5 | 6 | |

+---+---+---+

| 4 | 7 | 8 |

+---+---+---+

Move: L

+---+---+---+

| 1 | 2 | 3 |

+---+---+---+

| 5 | | 6 |

+---+---+---+

| 4 | 7 | 8 |

+---+---+---+

Move: L

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
|   | 5 | 6 |
+---+---+---+
| 4 | 7 | 8 |
+---+---+---+
```

Move: D

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
|   | 7 | 8 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
| 7 |   | 8 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
| 7 | 8 |   |
+---+---+---+
```

```
import random
```

```
# Calculate number of attacking pairs
```

```
def heuristic(state):
```

```
    conflicts = 0
```

```
    for i in range(4):
```

```
        for j in range(i + 1, 4):
```

```
            # Same row
```

```
            if state[i] == state[j]:
```

```
                conflicts += 1
```

```

        # Same diagonal
        if abs(state[i] - state[j]) == abs(i - j):
            conflicts += 1

    return conflicts

# Generate all possible neighboring states
def get_neighbors(state):
    neighbors = []

    for col in range(4):

        for row in range(4):

            if row != state[col]:

                new_state = state.copy()
                new_state[col] = row

                neighbors.append(new_state)

    return neighbors

# Hill Climbing
def hill_climbing(start):

    current = start
    current_h = heuristic(current)

    while True:

        neighbors = get_neighbors(current)

        # Find the best neighbor
        best = current
        best_h = current_h

        for neighbor in neighbors:

            h = heuristic(neighbor)

            if h < best_h:
                best = neighbor
                best_h = h

        # If no better neighbor exists
        if best_h >= current_h:

```

```

        break

    current = best
    current_h = best_h

    return current

# Print board
def print_board(state):

    for row in range(4):

        for col in range(4):

            if state[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")

        print()

# -----
# USER INPUT
# -----

print("4 Queens - Hill Climbing")

print("\nEnter initial state.")
print("Enter 4 numbers (0-3).")
print("Each number represents the row of the queen in each column.")

print("\nExample: 1 3 0 2")

start = list(map(int, input("\nInitial state: ").split()))

# Check input
if len(start) != 4 or any(x < 0 or x > 3 for x in start):

    print("Invalid input!")
    print("Enter 4 numbers between 0 and 3.")

else:

    print("\nInitial State:")
    print_board(start)

```

```

print("\nInitial heuristic:", heuristic(start))

# Apply Hill Climbing
solution = hill_climbing(start)

print("\nFinal State:")
print_board(solution)

print("\nFinal heuristic:", heuristic(solution))

if heuristic(solution) == 0:
    print("\nSolution found!")
else:
    print("\nHill Climbing got stuck at a local maximum.")

```

4 Queens - Hill Climbing

Enter initial state.

Enter 4 numbers (0-3).

Each number represents the row of the queen in each column.

Example: 1 3 0 2

Initial state: 1 1 0 2

Initial State:

```

. . Q .
Q Q . .
. . . Q
. . . .

```

Initial heuristic: 2

Final State:

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Final heuristic: 0

Solution found!